Chapter 4
Symmetric Cryptography

From Caesar to AES: why the classical ciphers fall, how a modern block cipher is built, and how to use one without leaking the picture

Information Security

Based on the Introduction to Cryptography lecture and the Breaking the Classics, AES and OpenSSL TPs

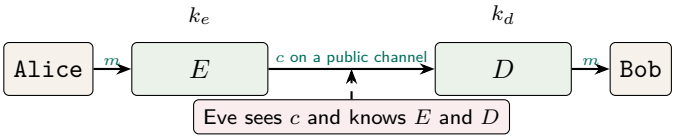
How to read this chapter

Chapters 1 to 3 were about what can go wrong on the web. This chapter is about the one tool that can make a channel private: a cipher. It is built so that each idea is defined before it is used. We start with what a cryptosystem is and what "secure" can even mean (Section 1). We then break three classical ciphers by hand, because each failure names a design requirement that modern ciphers satisfy (Sections 2 and 3). Those requirements, confusion and diffusion, lead to the round structure of DES and AES (Sections 4 to 6), each with one round computed by hand with the lecture's own numbers. A block cipher only handles 16 bytes, so Section 7 explains modes of operation and the famous ECB picture leak from the TP. Section 8 closes the loop with the question every symmetric cipher leaves open, how two parties agree on the key, which Diffie–Hellman answers and Chapter 5 completes. Section 9 maps the three TPs onto the chapter and Section 10 collects traps and classic exam problems. Every number in a worked example is one from the slides or the TPs, so you can check your hand computation against it.

1 What a Cipher Is, and What Secure Means

1.1 The objects

A **cryptosystem** is five things: a message space M, a ciphertext space C, a key space K, an encryption function $c = E(m, k_e)$ and a decryption function $m = D(c, k_d)$. It is **symmetric** when $k_e = k_d$, so whoever can encrypt can also decrypt, and **asymmetric** when the two keys differ and k_d cannot feasibly be derived from k_e (Chapter 5). **Cryptology** is the union of **cryptography**, designing such systems, and **cryptanalysis**, breaking them.



A cipher by itself promises only **confidentiality**: Eve learns nothing about m. Integrity (the message was not altered) and authentication (it really came from Alice) are separate goals with separate tools, and Chapter 5 shows that a cipher gives neither.

1.2 Kerckhoffs's principle

Key insight. Everything except the key is public. Design for an Eve who has your source code.

Eve in the picture knows E and D. That is not pessimism, it is the design rule stated by Auguste Kerckhoffs in 1883: a system must remain secure when everything about it except the key is public. Three practical consequences follow. Algorithms are published and reviewed, which is how DES and AES were chosen, through open competitions. A compromised key is replaced without redesigning anything. And a vendor's "secret proprietary algorithm" is a warning sign, because algorithms leak through reverse engineering and cannot be rotated the way keys can. Historically the balance was simple algorithms with long keys; today it is sophisticated public algorithms with short keys.

1.3 Attack models

"Secure" is meaningless until you say what the attacker has. The models are ordered by what the cryptanalyst holds besides the algorithm and the target ciphertext:

- **Ciphertext only** (COA): nothing more. Intercepted radio traffic.
- **Known plaintext** (KPA): some plaintext–ciphertext pairs under the same key. Encrypted files whose headers are standard (PDF magic bytes).
- **Chosen plaintext** (CPA): the attacker gets plaintexts of their choice encrypted. A web application that encrypts a cookie value the attacker supplied.
- **Chosen ciphertext** (CCA): the attacker gets ciphertexts of their choice decrypted, or at least learns something about the result. A server that reveals whether the padding of a submitted token was valid.

Power grows COA $<$ KPA $<$ CPA, CCA, and security under a stronger model implies security under the weaker ones. Modern ciphers are required to resist chosen-plaintext attacks because attackers routinely control inputs. Orthogonal to the model is the *method*: **analytical** attacks exploit a weakness of the algorithm or of its use; **brute force** tries every key and is a question of money (a cipher is **computationally secure** when the cost of cracking exceeds the value of the information); **social engineering** attacks the person; **implementation attacks** target the software, the hardware, side channels such as power traces, or the way the cipher is used.

Hard questions, answered

Q. Classify: power-trace analysis of a smart card; bribing the operator; trying all 2^{56} DES keys; exploiting the fact that ECB encrypts equal blocks equally.
A. Implementation attack; social engineering; brute force; analytical attack on the mode of use.

Q. Why is "computationally secure" an economic statement and "unconditionally secure" a mathematical one?
A. The first says cracking costs more than the information is worth, which depends on hardware prices; the second says the ciphertext carries no information about the plaintext at all, which only the one-time pad of Section 3 achieves.

2 Classical Ciphers, and Why Each One Falls

Three historical ciphers teach three lessons. A tiny key space falls to brute force. A huge key space is not enough when the statistics of the plaintext survive. A repeating key reduces to the previous case.

2.1 Caesar: too few keys

Number the letters 0..25. Caesar with key k encrypts $c = (p + k) \bmod 26$ and decrypts $p = (c - k) \bmod 26$. With $k = 3$, `Security is very interesting` becomes `VHFXULWB LU YHUB LQWHUHVWLQJ`. There are 26 keys, so an attacker who knows the cipher is Caesar does not need to find the key: they decrypt under all 26 and read the one that is English. The TP's warm-up is exactly this: `ESP VPJ TD FYVYZHY ECJ MCFEP QZCNP` with an unknown key is shift 11, `THE KEY IS UNKNOWN TRY BRUTE FORCE`.

2.2 Substitution: many keys, same statistics

A **simple substitution** replaces each letter by a fixed other letter: a permutation σ of the alphabet, $c = \sigma(p)$. The key space is $26! \approx 4 \times 10^{26}$, far beyond brute force. It falls anyway, and the reason is the lesson of the whole chapter. Count the letters of the ciphertext. For each plaintext letter a, the number of positions where the ciphertext shows $\sigma(a)$ is exactly the number of positions where the plaintext had a, because σ is one-to-one and applied position by position. So the multiset of counts is unchanged; only the labels moved. English counts are far from flat (E 12.7%, T 9.1, A 8.2, O 7.5, I 7.0, N 6.7, S 6.3, H 6.1, R 6.0, D 4.3, L 4.0, C 2.8; J, X, Q, Z rarest), so sorting the ciphertext counts largely determines σ, and short words, doubled letters and repeated patterns resolve the rest. The attack never enumerates keys, so the size of the key space is irrelevant.

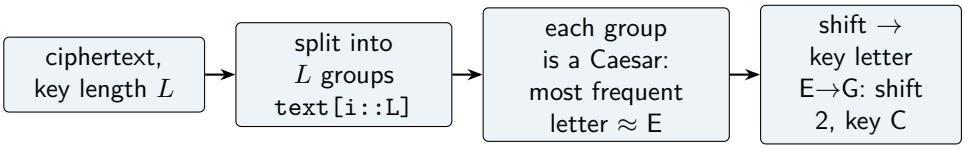
Key insight. A bijection applied letter by letter relabels the frequency histogram; it does not flatten it.

Worked example: The TP's letter wall

With the TP's table (A→Q, B→M, C→J, D→Z, E→T, ...), `KEEP THE SECRET SAFE` encrypts to `LTTN EKT YTJRTE YQGT` and `ERVYE DXVR SXFPJ` decrypts to `TRUST YOUR LOGIC`. For the unknown-key message, the hint order E > T > R > S/O > A > H > N > D > I > G > L is matched against the ciphertext ranking (T is the most frequent ciphertext letter, so T → E; then E → T, R → R, ...); the one-letter word `Q` must be A, and `EKT` with E,T known is THE. The recovered text says the Vigenère key has four letters, which is the clue for the next room. The slide's `GSRH RH Z HVXIVG` is the reversed alphabet (Atbash): `THIS IS A SECRET`.

2.3 Vigenère: a repeating key is several Caesars

Vigenère uses a keyword $k_0 k_1 \dots k_{L-1}$ that repeats: letter i is shifted by $k_{i \bmod L}$. The slide's `send reinforcements` under the key `somesecret` becomes `kszhjikejhjqqqwrvj`. The histogram is now flattened, because the same plaintext letter is shifted by different amounts, and for three centuries the cipher was called unbreakable. The attack is to undo the flattening:



Every letter in group i was shifted by the same k_i, so inside a group the cipher is Caesar and the most frequent ciphertext letter is almost certainly E. If L is unknown, Kasiski's observation finds it: a repeated trigram in the ciphertext usually comes from the same plaintext word hitting the same key position, so the distances between repeats are multiples of L. The attack needs the key to *repeat*. A random key as long as the message, used once, leaves one letter per group and nothing to count, which is the subject of the next section.

From the TP: Room 2

The TP ciphertext (`VVHWGQUIVY...`) with $L = 4$: `ciphertext[0::4]`, `[1::4]`, `[2::4]`, `[3::4]`; run the frequency program on each group, map the top letter to E, convert each shift to a key letter with the Vigenère table, and decrypt. The hidden access word is in the plaintext.

Hard questions, answered

Q. Why is a substitution cipher with 26! keys broken "faster" than Caesar is brute-forced?
A. Both are trivial, for different reasons: Caesar by 26 trials, substitution by analysis of surviving statistics. The point is that 26! bought nothing because the attack never enumerates keys.

Q. Vigenère with a 1000-letter random key on a 1000-letter message, used

once. Breakable by the group method?
A. No: $L = 1000$ gives one letter per group and no statistics. That is the one-time pad.

3 XOR and the One-Time Pad

Modern ciphers work on bits, and the letter shift becomes **XOR**: $0 \oplus 0 = 0$, $0 \oplus 1 = 1$, $1 \oplus 1 = 0$, "different gives 1". Its one property worth memorising is that it undoes itself, $(x \oplus k) \oplus k = x$, so a single operation both encrypts and decrypts. `Hello` XOR the repeating key `key` gives `00100011 00000000 00010101 00000111 00001010`; XOR with the key again returns `Hello`.

3.1 Perfect secrecy

Take a key that is uniformly random, as long as the message, and used once; encrypt by XOR. For a given ciphertext c and *any* candidate plaintext m of the same length there is exactly one key that maps one to the other, namely $k = m \oplus c$, and all keys were equally likely. So every plaintext is equally consistent with what Eve sees: the ciphertext carries no information at all. This is the **one-time pad**, the only unconditionally secure cipher. It is rarely used because its key must be as long as all traffic, truly random, delivered secretly and never reused; distributing the key is as hard as distributing the message.

3.2 What goes wrong with a repeating key

Drop any of the three conditions and the scheme is Vigenère on bytes, and it fails in every model. Under KPA, $m \oplus c = k$ hands over the key directly. Under CPA, encrypting an all-zero plaintext returns k as the ciphertext. Under CCA, decrypting an all-zero ciphertext returns k as the plaintext. Even under COA, the second ciphertext byte of the `Hello` example is `00000000`, which says the second key byte equals the second plaintext byte, and language structure does the rest. Reuse is the fatal case: $c_1 \oplus c_2 = m_1 \oplus m_2$ cancels the key entirely and two structured texts can be pulled apart (crib dragging). Block ciphers exist to get the convenience of a short key without any of this: a short key *generates* a long pseudorandom stream (CTR mode, Section 7).

Anatomy: the one-time pad, and every stream cipher

Part	What it does	If it is removed, weakened or misused
Key as long as the message	one key bit per message bit, no reuse of any bit	shorter key must repeat: Vigenère on bytes, group attack and KPA give the key
Key truly random	every plaintext equally consistent with c	a pseudo-random key with a short seed turns "perfect" into "as strong as the generator"; a biased key leaks the biased positions
Key used once	no relation between two ciphertexts	reuse: $c_1 \oplus c_2 = m_1 \oplus m_2$, crib dragging recovers both
XOR	self-inverse mixing	replacing XOR by a non-invertible operation loses decryption; by addition mod 256 changes nothing security-wise

4 Building a Block Cipher: Confusion, Diffusion, Rounds

Section 2 showed what a cipher must destroy: the structure of the plaintext. Shannon named the two properties that do it.

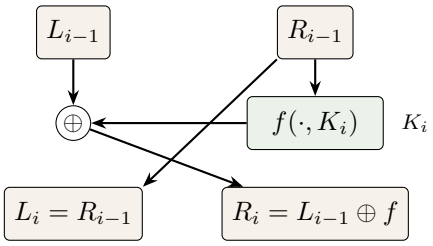
Concept: Confusion and diffusion

Confusion: the relation between key, plaintext and ciphertext is complex, which in practice means nonlinear. Caesar's $c = (p + k) \bmod 26$ has none: one known pair gives k. **Diffusion**: each input bit influences many output bits, so that local statistics (a frequent letter, a repeated word) spread out and vanish. Block ciphers obtain both by iterating **rounds**, each of which substitutes (confusion) and permutes or mixes (diffusion); one round is weak, ten are not.

4.1 The Feistel trick

A round must be invertible or the receiver cannot decrypt, yet the nonlinear function at its heart is easiest to design when it does not have to be. Horst Feistel's structure, used by DES, resolves this. Split the block into halves (L, R) and define one round as

$$L_i = R_{i-1}, \qquad R_i = L_{i-1} \oplus f(R_{i-1}, K_i).$$



Only the right half enters f. The left half is XOR-ed with the result and the halves swap. Decryption is the *same* round, because L_i is a copy of R_{i-1}, so the receiver can recompute $f(R_{i-1}, K_i) = f(L_i, K_i)$ and XOR it away.

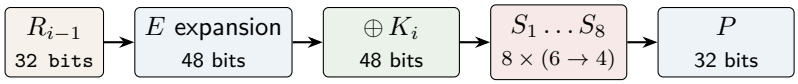
Inverting one round: $R_{i-1} = L_i$ and $L_{i-1} = R_i \oplus f(L_i, K_i)$. Apply from round n down to 1 and the whole cipher is undone by the encryption algorithm fed $K_n, \ldots, K_1$. This is why DES decrypts "for free" and why AES, which is not Feistel, needs an explicit inverse for each of its operations (Section 6).

Key insight. In a Feistel cipher, decryption is encryption with the subkeys in reverse order, and f never needs an inverse.

5 DES

5.1 The parameters

DES (1975) is the worked example of a Feistel cipher. Block 64 bits. Key 56 bits: the 64-bit input key carries 8 parity bits at positions 8, 16, ..., 64, which the first permuted choice PC-1 discards. Sixteen rounds, each with its own 48-bit subkey K_i. An initial permutation IP before round 1 and its inverse IP^{-1} after round 16, both fixed tables with no cryptographic role. The round function f takes the 32-bit right half and a subkey and returns 32 bits in four steps:



from the key schedule

1. **Expansion** E: 32 to 48 bits, by duplicating the two boundary bits of each 4-bit group so that each S-box sees its own 4 bits plus one from each neighbour.
2. XOR with the subkey K_i: the only place the key enters.
3. Eight **S-boxes**, each mapping 6 bits to 4 by table lookup. The *row* is given by the outer bits $b_1 b_6$, the *column* by the inner bits $b_2 b_3 b_4 b_5$. This is the only nonlinear step and therefore the source of all confusion; the slides call it crucial for DES security.
4. **Permutation** P: a fixed rearrangement that sends the four output bits of each S-box to four different S-boxes in the next round. This is the diffusion.

IP, E and P are position tables: they move bits and never change their values. Only the S-boxes compute anything.

Worked example: One round with the slide's numbers

Input block `0x0123456789ABCDEF`. IP's first row (58, 50, 42, 34, 26, 18, 10, 2) picks those input bits as the first output byte: `11001100`. After IP,

$R_0 = \texttt{0xF0AAF0AA}$. Expansion: $E(R_0) = \texttt{011110 100001 010101 010101 011110 100001 010101 010101}$. After XOR with K_1: $\texttt{0x6117BA866527}$. The first 6-bit group $\texttt{011000}$ has row $\texttt{00} = 0$ and column $\texttt{1100} = 12$, and $S_1[0, 12] = 5 = \texttt{0101}$. All eight boxes give $\texttt{0x5C82B597}$, and P yields $\texttt{0x234AA9BB}$, the output of f.
The toy round (4-bit halves, so you can do every step). $L_0 = \texttt{1010}$, $R_0 = \texttt{1001}$, $E' = [4, 1, 2, 3, 4, 1]$, $K_1 = \texttt{101011}$, $P' = [2, 4, 3, 1]$. $E'(R_0) = \texttt{110011}$ (bit 4, bit 1, bits 2, 3, 4, 1 of $\texttt{1001}$). XOR K_1: $\texttt{011000}$. S_1: row 0, column 12, output $\texttt{0101}$. P': $\texttt{1100}$. Then $L_1 = R_0 = \texttt{1001}$ and $R_1 = L_0 \oplus \texttt{1100} = \texttt{0110}$.

5.2 The key schedule

PC-1 keeps 56 of the 64 key bits and splits them into two 28-bit halves C_0, D_0. In round i each half is rotated left by one bit in rounds 1, 2, 9 and 16 and by two bits in the other twelve; PC-2 then selects a permuted 48-bit subset as K_i. The rotations total $4 \cdot 1 + 12 \cdot 2 = 28$, a full turn, so $C_{16} = C_0$ and $D_{16} = D_0$. Decryption uses the same sixteen subkeys in the order $K_{16}, \ldots, K_1$ (the Feistel trick). Equivalently, run the schedule backwards: no rotation in round 1, one bit to the *right* in rounds 2, 9, 16, two bits right elsewhere. No new key material is ever generated.

Worked example: The toy key schedule

Key $\texttt{10101101}$, PC-1 $= [1, 2, 3, 5, 6, 7]$ keeps $\texttt{101110}$; $C_0 = \texttt{101}$, $D_0 = \texttt{110}$. Round 1 rotates by 1: $C_1 = \texttt{011}$, $D_1 = \texttt{101}$; PC-2 $= [1, 2, 4, 5]$ applied to $\texttt{011101}$ gives $K_1 = \texttt{0110}$. Round 2 continues from C_1, D_1, never from the master key: $C_2 = \texttt{110}$, $D_2 = \texttt{011}$; PC-2 on $\texttt{110011}$ gives $K_2 = \texttt{1101}$.

5.3 Why DES died, and what replaced it

History. The EFF's Deep Crack, a $250,000 machine, found a DES key in 56 hours in 1998.

The key space is $2^{56} \approx 7.2 \times 10^{16}$. At 10^9 keys per second one machine needs about 7.2×10^7 seconds, 2.3 years; a purpose-built parallel machine did it in two days in 1998. DES died of key size, not of cryptanalysis: no practical analytical attack ever beat brute force. The obvious repair, **double DES** with two keys, does not help: **meet in the middle** encrypts a known plaintext under all 2^{56} values of K_1, decrypts the matching ciphertext under all 2^{56} values of K_2, and looks for a collision between the two tables, about 2^{57} work and a lot of memory instead of 2^{112}. **Triple DES** applies DES three times in encrypt–decrypt–encrypt order (so that $K_1 = K_2 = K_3$ degrades to single DES for compatibility) with two keys (112 bits effective) or three (168). It survives in legacy finance; AES replaced it and runs about five times faster in hardware.

Anatomy: DES, part by part

Part	What it does	If it is removed, weakened or misused
IP, IP^{-1}	fixed bit shuffles at the ends	no security role; removing them changes nothing
Feistel split and swap	makes any f invertible; decryption = encryption with reversed subkeys	no swap: the left half is never processed by f and leaks through; a non-Feistel f with no inverse cannot decrypt
Expansion E	48 bits so each S-box also sees a bit from each neighbour	without the overlap, S-boxes act on disjoint 4-bit groups: diffusion only via P, slower avalanche
$\oplus K_i$	the only entry of the key	no key in some rounds: those rounds are public and can be peeled off for free
S-boxes	the only nonlinear step; confusion	linear or affine S-boxes make DES a linear map: a few known pairs and Gaussian elimination recover the key; a single S-box reused eight times, or one that is far from uniform, enables differential/linear attacks
Permutation P	spreads each S-box's 4 output bits to 4 different S-boxes next round; diffusion	identity P: each 4-bit lane evolves on its own; attack each lane separately, $8 \times 2^{\text{small}}$ instead of 2^{56}
16 rounds	enough iterations for full avalanche and against differential cryptanalysis	few rounds (1–4): known-plaintext attacks far below brute force; 1 round: f output is $R_1 \oplus L_0$, read off directly
Key schedule (rotations, PC-2)	distinct subkeys per round	identical subkeys ($K_i = K$ for all i): slide attacks; weak keys where the schedule is constant make encryption its own inverse
56-bit key	key space 2^{56}	too small for 1998 hardware; the actual cause of death. 2DES: meet in the middle, 2^{57}. 3DES EDE: 2^{112} effective

Go deeper

- S-box indexing: outer bits row, inner bits column. Row 2 of S_1 is 4 1 14 8 13 6 2 11 15 12 9 7 3 10 5 0, so S_1(`100110`): row `10` = 2, column `0011` = 3, output 8 = `1000`.
- Timeline: DES 1975 (56-bit key, 64-bit block); 3DES 1995 (112/168, 64); Twofish, Serpent, Rijndael 1998 (128/192/256 keys, 128-bit block); AES standard 2001 (FIPS-197).
- Why 56 bits from a 64-bit key: the eight parity bits are dropped by PC-1; they were an error check for key entry, never secret material.

6 AES

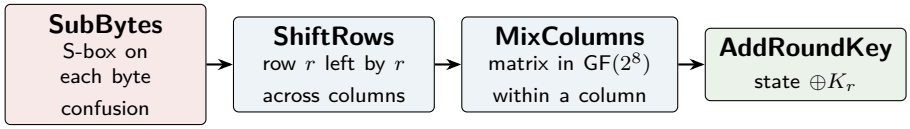
AES is the other design family, a **substitution–permutation network**: every operation acts on the whole block, so every operation needs an explicit inverse. Before the four operations we need the arithmetic they live in.

6.1 Bytes as field elements

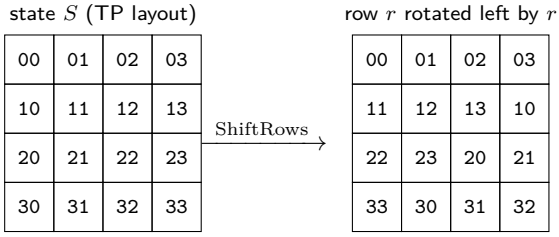
A byte is read as a polynomial of degree below 8 with coefficients 0 or 1: `57` = `01010111` = $x^6 + x^4 + x^2 + x + 1$. Addition is XOR. Multiplication is polynomial multiplication reduced modulo $m(x) = x^8 + x^4 + x^3 + x + 1$ (`0x11B`). This is the field GF(2^8), and the one fact to carry is how to multiply by `02` ("xtime"): shift left one bit, and if the bit shifted out was 1, XOR with `1B`, because $x^8 \equiv x^4 + x^3 + x + 1$. Then $\texttt{03} \cdot b = (\texttt{02} \cdot b) \oplus b$. Every nonzero byte has a multiplicative inverse, which is what makes SubBytes and MixColumns invertible.

6.2 The state and the four operations

AES-128 (Rijndael, by Rijmen and Daemen; FIPS-197, 2001) encrypts a 128-bit block held as a 4×4 byte **state**, filled column by column: input byte b_{i+4j} sits at row i, column j. Keys of 128, 192 or 256 bits give 10, 12 or 14 rounds and $N_r + 1$ round keys. Encryption is AddRoundKey(K_0), then $N_r - 1$ full rounds of SubBytes, ShiftRows, MixColumns, AddRoundKey, then a final round without MixColumns.



- **SubBytes** replaces each byte by its S-box entry (`00` → `63`, `01` → `7C`, `19` → `D4`, `71` → `A3`). It changes values, never positions. It is the only nonlinear step: remove it and AES is a linear map, solvable from a few known pairs.
- **ShiftRows** rotates row r left by r positions. It moves bytes *between* columns.
- **MixColumns** multiplies each column by a fixed matrix in GF(2^8). It mixes bytes *within* a column: changing one input byte changes all four output bytes.
- **AddRoundKey** XORs the state with the round key. It is the only step that depends on the key.



MixColumns then acts on each column of the shifted state, so after one round a byte has touched its whole column and the row shift has carried it into the other columns. After two rounds every byte depends on every input byte: full diffusion.

Worked example: MixColumns on the TP column `D4 BF 5D 30`

The first output byte is $b_0 = (\texttt{02} \cdot \texttt{D4}) \oplus (\texttt{03} \cdot \texttt{BF}) \oplus \texttt{5D} \oplus \texttt{30}$.

- $\texttt{02} \cdot \texttt{D4}$: `11010100` shifted left is `10101000` = `A8`; a 1 fell off, so XOR `1B`: `B3`.
- $\texttt{02} \cdot \texttt{BF}$: `10111111` shifted is `01111110` = `7E`, carry, XOR `1B`: `65`. Then $\texttt{03} \cdot \texttt{BF} = \texttt{65} \oplus \texttt{BF} = \texttt{DA}$.
- $\texttt{B3} \oplus \texttt{DA} = \texttt{69}$; $\texttt{69} \oplus \texttt{5D} = \texttt{34}$; $\texttt{34} \oplus \texttt{30} = \texttt{04}$.

The full column becomes `04 66 81 E5`; AddRoundKey with a key byte `A0` turns `04` into `A4`. The TP's all-`0F` round key XORs every byte with `0F` (`23` → `2C`), and its SubBytes of the first row `00 01 02 03` is `63 7C 77 7B`.

6.3 Key expansion and decryption

AES-128 expands the key into 44 four-byte words; $w_0..w_3$ are the key itself. For $i \geq 4$, take $t = w_{i-1}$; if i is a multiple of 4, rotate t by one byte (RotWord), push each byte through the S-box (SubWord) and XOR the round constant $\texttt{Rcon}[i/4]$ into its first byte; then $w_i = w_{i-4} \oplus t$. Round key r is $w_{4r}..w_{4r+3}$. The round constants are the successive powers of `02` in the field, `01 02 04 08 10 20 40 80 1B 36`, which is why `1B` follows `80`.

Decryption runs the inverses in reverse order: AddRoundKey(K_{10}); for rounds 9 down to 1, InvShiftRows (rotate right), InvSubBytes (inverse S-box), AddRoundKey, InvMixColumns (matrix `0E 0B 0D 09`); finally InvShiftRows, InvSubBytes, AddRoundKey(K_0). AddRoundKey is its own inverse because XOR cancels. This is the engine the TP's Act II asks you to rebuild.

From the TP: AES without diffusion

Remove ShiftRows and MixColumns from a two-round cipher on one byte: $c = S(S(p \oplus k_0) \oplus k_1) \oplus k_2$. From the pair (`00`, `A4`), $k_2 = \texttt{A4} \oplus S(S(k_0) \oplus k_1)$ is a function of (k_0, k_1). Enumerate the 2^{16} pairs, derive k_2 for each, keep the candidates that also map `01` → `F8` and `02` → `3F`. Each byte position of a 16-byte block could be attacked *independently* this way with three known plaintexts, 16×2^{16} work. With diffusion, every output byte depends on all sixteen input bytes and the state must be attacked as a whole, 2^{128}. Confusion without diffusion is attackable byte by byte; that is the lesson of the TP and the reason AES has four operations and not one.

Go deeper

- The last round omits MixColumns because a trailing public linear map adds no security, and dropping it keeps encryption and decryption structurally symmetric.
- AES-128 versus AES-256: key length, rounds (10 versus 14), round keys (11 versus 15) and the expansion details change; the block size and the four operations do not.
- $\texttt{02} \cdot \texttt{57}$: $\texttt{01010111} \rightarrow \texttt{10101110} = \texttt{AE}$, no carry. $\texttt{03} \cdot \texttt{57} = \texttt{AE} \oplus \texttt{57} = \texttt{F9}$.

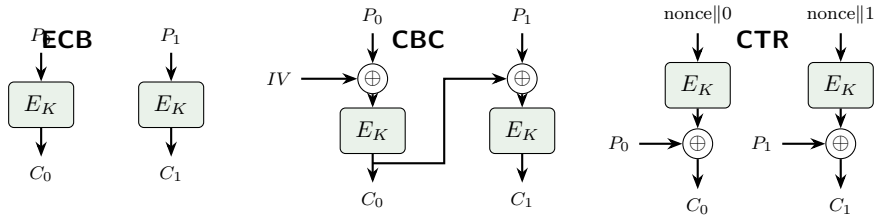
Anatomy: AES, part by part

Part	What it does	If it is removed, weakened or misused
AddRoundKey (K_0 first)	the only key-dependent step; whitening before round 1	no initial AddRoundKey: round 1 is a public function of the plaintext, peel it off; no key in a round: that round is free for the attacker
SubBytes (S-box)	the only nonlinearity; confusion; built from inversion in GF(2^8)	identity or affine S-box: the whole cipher is linear, solved from a few known pairs; a weak S-box (not bijective) loses decryption; a low-degree one invites algebraic attacks
ShiftRows	moves bytes *between* columns	without it, each column evolves independently: attack 4 columns separately, 4×2^{32} instead of 2^{128}
MixColumns	mixes bytes *within* a column; every output byte depends on all four	without it, each byte position evolves independently: the TP's byte-wise 2^{16} attack; without both diffusion steps, 16 separate 8-bit ciphers
10/12/14 rounds	full diffusion after 2 rounds, security margin beyond	1–2 rounds: trivially broken; 4–6 rounds: academic attacks; the margin is what the rounds buy
Key expansion (RotWord, SubWord, Rcon)	different round keys, nonlinear in the master key	all round keys equal: slide and related-key attacks; linear expansion (no SubWord): related-key attacks improve
Final round without MixColumns	keeps encryption/decryption symmetric	adding it changes no security; it is a public linear map
GF(2^8) with `1B`	invertible byte arithmetic	another reduction polynomial still works if irreducible; a reducible one has zero divisors, MixColumns stops being invertible

7 Modes of Operation

A block cipher encrypts 16 bytes. A **mode** says how to encrypt a message of many blocks, and the choice of mode decides whether a strong cipher produces

a strong system.



- **ECB** (electronic codebook): $C_i = E_K(P_i)$. Parallel in both directions, random access. Deterministic per block: equal plaintext blocks give equal ciphertext blocks, so the *structure* of the data leaks even though no block is decrypted.
- **CBC** (cipher block chaining): $C_i = E_K(P_i \oplus C_{i-1})$ with $C_0 = IV$, and $P_i = D_K(C_i) \oplus C_{i-1}$. Encryption is sequential (it needs C_{i-1}); decryption is parallel and random-access (it needs only C_i and C_{i-1}). The **initialisation vector** is random and sent in clear; it is not secret, it only has to be fresh so that the same message encrypted twice gives different ciphertexts.
- **CTR** (counter): $C_i = P_i \oplus E_K(\text{nonce}\|i)$. The block cipher generates a key stream and the mode is a stream cipher: fully parallel, random access, no padding. The nonce must never repeat under one key, or two messages share a pad and Section 3's reuse attack applies.

Others exist (CFB, OFB), and **CCM** combines CTR for confidentiality with CBC-MAC for integrity, the authenticated encryption that Chapter 5 will show is needed.

7.1 Error propagation in CBC

$P_i = D_K(C_i) \oplus C_{i-1}$ pushes a corrupted C_i through D_K, whose output is then pseudorandom garbage. $P_{i+1} = D_K(C_{i+1}) \oplus C_i$ XORs the corrupted C_i directly, bit for bit, so exactly the flipped bits flip. P_{i+2} depends only on C_{i+1} and C_{i+2} and is intact. This self-healing is also an attack: without an integrity check, Eve can flip chosen bits of P_{i+1} by flipping them in C_i. Confidentiality is not integrity.

Key insight. In CBC a corrupted ciphertext block garbles one plaintext block and flips the same bits in the next; nothing after that is affected.

Anatomy: modes, part by part

Part	What it does	If it is removed, weakened or misused
Block cipher E_K	the one secure primitive; a pseudorandom permutation of 16 bytes	any mode is only as good as E_K; but a perfect E_K in ECB still leaks
ECB: independence of blocks	parallel, random access	equal blocks give equal ciphertexts: structure leaks (the image); blocks can be cut, reordered, replayed
CBC: chaining $P_i \oplus C_{i-1}$	each block depends on all previous ones; equal blocks differ	without chaining it is ECB; chaining with the previous *plaintext* (PCBC) or a constant instead of C_{i-1} reintroduces patterns
CBC: random IV	same message twice gives different ciphertexts	fixed IV: the first block is ECB; predictable IV under CPA: an attacker tests guesses block by block (the BEAST idea); secret IV: unnecessary
CTR: nonce ‖ counter	unique input per block gives a fresh key stream	nonce reused under one key: two-time pad; counter that wraps or overlaps between messages: same
Padding (PKCS#7)	fills the last block	a server that reports "bad padding" is a CCA oracle (padding oracle attack on CBC)
Integrity tag (CCM, GCM, HMAC)	detects modification	CBC without a MAC: flip bits of P_{i+1} through C_i; CTR without a MAC: flip any bit of any plaintext directly

From the TP: Break the image, not AES

OpenSSL's salted format begins with the 8-byte signature `Salted__` and an 8-byte salt (public: the key is derived from the password and the salt with PBKDF2), so the first 16 bytes of `classified.enc` are not ciphertext. The image is $432 \times 475 = 205{,}200$ bytes, exactly 12,825 blocks, so PKCS#7 appends one full padding block: 205,216 ciphertext bytes, 205,232 in the file. Count equal 16-byte blocks (`collections.Counter`), give equal blocks the same grey shade, and the outline of the picture appears: a *pattern, not the secret*, since no pixel value was recovered. Decrypt with `openssl enc -d -aes-256-ecb -pbkdf2 -p -in classified.enc -out recovered.raw` and the Mission 1 password. Re-encrypt with `-aes-256-cbc`: essentially no repeated blocks, because each block is XOR-ed with the previous ciphertext before encryption.

Hard questions, answered

Q. Why is a clear-text IV not a weakness, and what would be?
A. The IV only has to be unpredictable or unique so that equal first blocks differ; knowing C_0 does not help invert E_K. Reusing an IV, or using a

predictable one against a chosen-plaintext attacker, is the weakness.

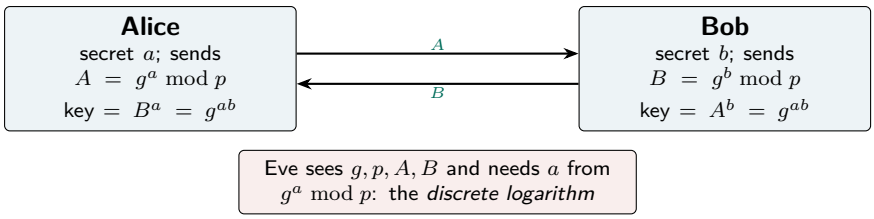
Q. Rank ECB, CBC, CTR on encryption parallelism, decryption parallelism, random access, pattern hiding.
A. ECB yes/yes/yes/no. CBC no/yes/yes/yes. CTR yes/yes/yes/yes, given a unique nonce.

Q. AES-256-ECB on a column of 16-byte customer IDs. Strong cipher; is the system strong?
A. No. Equal IDs give equal ciphertexts, so frequency and equality analysis work without touching AES. Use CBC or CTR with fresh IVs, or authenticated encryption.

8 Agreeing on the Key: Diffie–Hellman

Every cipher so far assumes Alice and Bob already share K, and delivering it needs a secure channel, which is the problem the cipher was supposed to solve. Diffie–Hellman breaks the circle. Public parameters: a prime p and a generator g. Alice picks a secret a and sends $A = g^a \bmod p$; Bob picks b and sends $B = g^b \bmod p$. Alice computes $B^a = g^{ab}$, Bob computes $A^b = g^{ab}$: the same number, from what each received and their own secret.

History. Merkle proposed public-key ideas as a Stanford student in 1974 and was told it was "extremely dangerous to transmit key information in the clear". Diffie and Hellman published in 1976.



Eve sees g, p, A, B and would need a from $A = g^a \bmod p$, the **discrete logarithm** problem. For $p = 13$ it is a table lookup (the slide's $6^x \bmod 13 = 2$ has $x = 5$, and $= 9$ has $x = 4$); for a 2048-bit p no known algorithm finishes. What DH does *not* give is authentication: nothing tells Alice that B came from Bob. A **man in the middle** runs one exchange with Alice (key k_1) and another with Bob (key k_2) and relays, decrypting and re-encrypting everything, so "Transfer $10" can become "Transfer $100" on the way. The endpoints must be authenticated, which is where Chapter 5 begins, with signatures and certificates.

Hard questions, answered

Q. Which hard problem underlies DH and which underlies RSA?
A. Discrete logarithm; integer factorisation.

Q. Does encrypting the DH messages with Bob's public key stop the man in the middle?
A. Only if Alice already holds Bob's *authentic* public key. If Eve can substitute hers, she decrypts and relays. The problem is key authenticity,

hence certificates.

Anatomy: Diffie–Hellman, part by part

Part	What it does	If it is removed, weakened or misused
Large prime p	the group $\mathbb{Z}_p^*$ in which discrete logs are hard	small p: table lookup; a composite or smooth $p-1$: Pohlig–Hellman solves the log in pieces
Generator g	g^a ranges over a large subgroup	g of small order (e.g. $g=1$ or $g=p-1$): only a few possible keys, Eve tries them all
Secrets a, b random and large	unpredictable exponents	small or guessable a: brute-force g^a; reused a across sessions: no forward secrecy
Exchange of g^a, g^b only	Eve must solve a discrete log	sending a itself, or g^{ab}, is the end; sending g^a and g^b over an unauthenticated channel invites the man in the middle
Authentication (signatures, certificates)	binds g^a to Alice	absent in plain DH, hence MITM: Eve shares k_1 with Alice and k_2 with Bob and relays
Key derivation (hash of g^{ab})	turns a group element into a uniform AES key	using g^{ab} bits directly: some bits are biased or predictable

9 TP Companion

Technique: The three TPs, mapped onto the chapter

- **Breaking the Classics** (Section 2). Warm-up: Caesar encrypt/decrypt and a 26-key brute force loop. Room 1: substitution by table, then a frequency counter (ignore non-letters, count, percentages, sort) and a partial mapping refined by short words and repeats. Room 2: Vigenère with $L = 4$ via `text[i::4]`, the counter per group, top letter $\rightarrow$ E, shift $\rightarrow$ key letter. Keep every program; the deliverable includes the code.
- **Mission AES** (Section 6). Act I by hand: AddRoundKey with the all-`0F` key; SubBytes of row 0 with the S-box extract; ShiftRows of the index state; MixColumns byte $b_0 =$ `04`; the "AES without diffusion" 2^{16} search. Act II in code: `add_round_key` (XOR), `inv_shift_rows` (right rotations 0, 1, 2, 3), `inv_sub_bytes` (invert the S-box list), `gf_mul` via `xtime`, `inv_mix_columns` with `0E 0B 0D 09`, and `decrypt_block` in the order AddRoundKey(K_{10}), rounds 9..1 of InvShiftRows, InvSubBytes, AddRoundKey, InvMixColumns, then InvShiftRows, InvSubBytes, AddRoundKey(K_0).
- **OpenSSL, Mission 2** (Section 7). `xxd -l 64 classified.enc` shows `Salted__` and the salt; skip 16 bytes; `Counter` over 16-byte slices; render equal blocks as equal shades with Pillow; decrypt with

`-aes-256-ecb -pbkdf2`; rebuild the 432×475 image; re-encrypt with `-aes-256-cbc` and compare repeated-block counts. Missions 1 and 3 (RSA) are in Chapter 5.

10 Tricks, Techniques and Classic Problems

Technique: Reading a "custom cipher" on the exam

When the problem defines its own cipher (a modified DES, an AES without a step, a home-made mode), walk this list in order; the first item that fails is usually the intended flaw.

1. **Where does the key enter?** Find every operation that uses K. Everything before the first key use and after the last is public and can be stripped by the attacker.
2. **Is there any nonlinearity?** Only S-boxes and modular multiplication are nonlinear; XOR, shifts, permutations, addition of constants are linear. A cipher with no nonlinear step is solved by linear algebra from a few known plaintexts, whatever the key size.
3. **Does every output bit depend on every input bit and every key bit?** Trace one byte through a round. If some lanes never mix, attack each lane separately and multiply, not exponentiate, the work.
4. **How many rounds, and are the round keys different?** One or two rounds, or identical round keys, give direct or slide attacks.
5. **Is encryption deterministic?** No IV, no nonce, no random padding: equal inputs give equal outputs, and the attacker tests guesses by encrypting them (ECB, textbook RSA).
6. **Is anything reused?** A key stream, a nonce, an IV, a one-time pad: XOR the two ciphertexts.
7. **Are the parameters big enough?** Key bits, modulus bits, p for DH; small means brute force or trial division; and remember meet in the middle for double encryption.
8. **Is integrity checked?** If not, which bits can the attacker flip in the plaintext by editing the ciphertext, and does the receiver act on them?

Warning / Gotcha: Common student mistakes

- **"A huge key space means secure."** Substitution has 26! keys and falls to counting letters. Key space defends against brute force only.
- **Reading the S-box with the wrong bits.** Outer bits b_1b_6 are the row, inner $b_2b_3b_4b_5$ the column.
- **Restarting the DES key schedule from the master key each round.** C_i, D_i continue from C_{i-1}, D_{i-1}.
- **Forgetting the `1B` reduction.** Multiplying by `02` is a shift *and* an XOR with `1B` when a 1 falls off the top.
- **Saying DES and AES both decrypt with reversed keys.** Only Feistel ciphers do; AES needs InvSubBytes, InvShiftRows, InvMix-

Columns.

- **Treating the IV or the salt as a secret.** Both are public; they must be fresh (IV) or per-file (salt), not hidden.
- **Blaming AES for the ECB penguin.** The cipher is fine; the mode leaks equality of blocks.
- **Believing DH authenticates.** It gives a shared secret with an unknown party; without authentication a man in the middle holds both keys.

Worked example: Classic exam problems

P1. A 1000-letter English ciphertext is monoalphabetic substitution. Sketch the attack and say why 26! does not matter.
Count letters, map the top counts to E, T, A, O, I, N, use one-letter and three-letter words and doubled letters to fix the rest. The attack reads statistics, never enumerates keys; the key space bounds only brute force.

P2. Vigenère, key length 5, 2000 letters. Work needed?
Five groups of 400 letters; each is a Caesar and its top letter is E with overwhelming probability. Five subtractions. If L is unknown, Kasiski on repeated trigrams or index of coincidence over candidate L first.

P3. The same one-time pad key encrypts two messages. What does Eve get?
$c_1 \oplus c_2 = m_1 \oplus m_2$; guessing a word in one message (a crib) reveals the aligned fragment of the other.

P4. Toy Feistel round with $f(R, K) = R \oplus K$, $L_0 = \texttt{1100}$, $R_0 = \texttt{1010}$, $K_1 = \texttt{0110}$. Output, and how to invert.
$f = \texttt{1010} \oplus \texttt{0110} = \texttt{1100}$; $L_1 = \texttt{1010}$, $R_1 = \texttt{1100} \oplus \texttt{1100} = \texttt{0000}$. Invert: $R_0 = L_1$, $L_0 = R_1 \oplus f(L_1, K_1) = \texttt{0000} \oplus \texttt{1100} = \texttt{1100}$.

P5. S_1 on 011011.
Row $\texttt{01} = 1$, column $\texttt{1101} = 13$; look up $S_1[1, 13]$ in the table (row 1: 0 15 7 4 14 2 13 1 10 6 12 11 9 5 3 8, so $5 = \texttt{0101}$).

P6. Compute $\texttt{02} \cdot \texttt{80}$ and $\texttt{03} \cdot \texttt{80}$ in GF(2^8).
10000000 shifted is 00000000 with a carry, XOR 1B: 1B. Then $\texttt{03} \cdot \texttt{80} = \texttt{1B} \oplus \texttt{80} = \texttt{9B}$.

P7. A 1 MB file is encrypted in CBC and byte 5000 of the ciphertext is corrupted in transit. What decrypts wrongly?
Block $\lfloor 5000/16 \rfloor = 312$ entirely, and the corresponding byte of block 313; everything else is correct.

P8. Why does the planner of a secure protocol pair DH with a signature?
DH gives secrecy against a passive Eve but no authentication; signing the DH values with a key whose public half is certified proves who sent them and defeats the man in the middle.

Summary: Chapter 4 in six lines

1. Only the key is secret (Kerckhoffs); security is stated against a model, COA $<$ KPA $<$ CPA/CCA.
2. Caesar has too few keys, substitution keeps the letter statistics, Vigenère is L Caesars; the one-time pad is perfect but its key is the problem.
3. Confusion (nonlinear S-boxes) and diffusion (permutation and mixing) over many rounds destroy plaintext structure; Feistel makes any f invertible for free.
4. DES: 64-bit block, 56-bit key, 16 rounds of E, XOR, S-boxes, P; killed by brute force in 1998; 2DES falls to meet in the middle, 3DES survives.
5. AES: bytes in GF(2^8) with `1B`; SubBytes, ShiftRows, MixColumns, AddRoundKey for 10/12/14 rounds; explicit inverses.
6. ECB leaks structure, CBC chains with a public fresh IV, CTR is a parallel stream; DH agrees a key over a public channel but authenticates nobody.